EXCELLENCE AND SERVICE

Department of Computer Science

CHRIST (Deemed to be University)

Bangalore Central Campus • Hosur Road, Bengaluru - 560029

Subject

Full Stack Development Lab

PRACTICAL ASSIGNMENT

Exercise-3: Demonstrate Event Handling using JavaScript

Domain: **CampusVibe — Christ University Event & Club Hub**

STUDENT CREDENTIALS

Anushya C

Register Number: **2648508**

Degree: MCA / M.Sc Computer Science

1 OBJECTIVE & PROBLEM STATEMENT

Objective: To understand and implement JavaScript event handling by developing an interactive webpage that demonstrates at least three unique events and performs meaningful actions based on user interaction.

Problem Statement: Develop a responsive webpage and demonstrate the handling of a minimum of three different JavaScript events. The selected events should serve a clear purpose and improve user interaction or usability.

Evaluation Note: *The* `click` *event was excluded from the mandatory 3 unique events as per slide rubric. This implementation demonstrates* ***12 unique, non-click events*** *integrated seamlessly within the student's* ***CampusVibe (*** `fsdlab2.html` ***)*** *project.*

2 IMPLEMENTED EVENTS MATRIX (12 UNIQUE EVENTS)

#	EVENT NAME	TARGET ELEMENT	FUNCTIONAL PURPOSE	DYNAMIC FEEDBACK & ACTION
1	`load`	`window`	Welcome user when page loads & init metrics	Native alert: *"Welcome to CampusVibe — Christ University Event & Club Hub!"*
2	`input`	`#eventDemoInput`	Real-time dynamic typing preview	Live renders `You typed: [query]` with character count & clear button
3	`resize`	`window`	Detect live browser viewport width changes	Updates `Current Screen Width : [X] px` with responsive breakpoint badge
4	`mouseover`	`.event-card, .club-card`	Highlight cards upon pointer entry	Card elevates with `-translate-y-2`, shadow-2xl, blue ring glow
5	`mouseout`	`.event-card, .club-card`	Restore original card appearance	Reverts elevation and border glow smoothly via transition properties
6	`focus`	`#eventDemoInput, form inputs`	Signal active keyboard focus	Glows with blue ring and updates pill: `State: Active (focus)`
7	`blur`	`#eventDemoInput, form inputs`	Handle focus dismissal and validation	Updates status pill to `State: Inactive (blur)`
8	`change`	`#domainTrackSelect`	Switch campus event tracks dynamically	Instantly renders track schedule, venue, eligibility, and duration badges
9	`submit`	`#rsvpForm`	Intercept form submission & validate	`e.preventDefault()` stops reload; checks inputs, writes LocalStorage & Canvas
10	`dblclick`	`.event-card, .club-card`	Interactive quick star / bookmark action	Toggles amber background highlight and shows celebratory toast notification
11	`scroll`	`window`	Track page reading progression	Calculates percentage and expands top gradient bar from 0% to 100%
12	`contextmenu`	`#event-lab`	Custom contextual action palette	`e.preventDefault()` displays custom floating popup menu at cursor coordinates

3 EVENT INTEGRATIONS & VISUAL DOCUMENTATION

Figure 1: Window Load Alert ('load')

Slide 7 Spec

Popup Dialog:

"This page says:
Welcome to CampusVibe – Christ University Event & Club Hub!"

Fires automatically when the window and external assets (Tailwind CDN, Font Awesome, images) finish loading via `window.addEventListener('load')`.

Figure 2: Real-time 'input' & 'resize'

Slide 7 Spec

You typed: HackTheCampus

Current Screen Width : 1280 px [Desktop]

The `input` event streams typed characters instantaneously with character count, while `resize` updates the exact pixel width and breakpoint pill.

Figure 3: Card Hover ('mouseover'/'mouseout')

Slide 6 Spec

HackTheCampus 2026 (Hover State)

Transform: `-translate-y-2` | Ring: `ring-2 ring-blue-500`

Hovering adds elevation and vibrant glow; pointer departure seamlessly restores the neutral background.

Figure 4: Real-time Event Console

Viva Feature

```
[LOAD] Window resources initialized
[INPUT] Query: "Hackathon" (Length: 9)
[RESIZE] Viewport: 1280px x 800px
[MOUSEOVER] Hovering card-hackathon
```

Streams every event dispatch in real-time, proving execution clarity for faculty viva evaluation.

4 GITHUB REPOSITORY LINK (DELIVERABLE 3)

OFFICIAL REPOSITORY URL:

https://github.com/Anushyaaaa/Full-Stack-Development

Branch: main • Up to date

File: **fsdlab2.html**

Target: **Exercise-3 Event Handling**

Author: **Anushya C**

5 CORE SOURCE CODE (JAVASCRIPT EVENT LISTENERS)

```
// 1. Window 'load' Event
window.addEventListener('load', function() {
    updateScreenWidth();
    setTimeout(() => alert("Welcome to CampusVibe — Christ University Event & Club Hub!"), 400
});

// 2. Input 'input' Event (Live dynamic typing)
demoInput.addEventListener('input', function(e) {
    const val = e.target.value;
    typedValue.textContent = val || '(start typing above...)';
    charCount.textContent = val.length;
    logEvent('INPUT', `Search: "${val}"`);
});

// 3. Window 'resize' Event (Live browser width counter)
window.addEventListener('resize', function() {
    updateScreenWidth();
    logEvent('RESIZE', `${window.innerWidth}px x ${window.innerHeight}px`);
});

// 4 & 5. 'mouseover' & 'mouseout' Events (Feature / Event Cards)
document.querySelectorAll('.event-card, .club-card').forEach(card => {
    card.addEventListener('mouseover', () => card.classList.add('ring-2', 'ring-blue-500', '-t
    card.addEventListener('mouseout', () => card.classList.remove('ring-2', 'ring-blue-500', '
    card.addEventListener('dblclick', () => card.classList.toggle('bg-amber-50/50'));
});

// 6 & 7. 'focus' & 'blur' Events (Input active status)
demoInput.addEventListener('focus', () => fieldFocusStatus.textContent = 'State: Active (focus
demoInput.addEventListener('blur', () => fieldFocusStatus.textContent = 'State: Inactive (blur

// 8. Select 'change' Event (Dynamic domain preview)
domainTrackSelect.addEventListener('change', e => renderTrackPreview(e.target.value));

// 9. Window 'scroll' Event (Dynamic progress bar)
window.addEventListener('scroll', () => {
    const p = (window.scrollY / (document.body.scrollHeight - window.innerHeight)) * 100;
    document.getElementById('scrollProgressBar').style.width = p + '%';
});

// 10. 'contextmenu' Event (Custom right-click action palette)
demoSection.addEventListener('contextmenu', e => {
    e.preventDefault();
    openCustomMenu(e.pageX, e.pageY);
});
```

6 AI USAGE DECLARATION (MANDATORY DELIVERABLE)

AI Tools Utilized: Google Antigravity (Powered by Google DeepMind Advanced Agentic Coding & Gemini 3.8 Flash).

Purpose of AI Assistance:

- Designing responsive Tailwind CSS component layouts mirroring the Christ University Exercise-3 slide requirements.
- Structuring robust, performant DOM event listeners (`load` , `input` , `resize` , `mouseover` , `mouseout` , `change` , `scroll` , `contextmenu`).
- Structuring an asynchronous, non-blocking real-time event activity console.
- Formatting the academic submission documentation adhering to the 10-mark evaluation rubrics.

Declaration: All architectural patterns, event integrations, and UI components were reviewed, tested, and validated by the student in the local environment.

7 REFLECTIONS & VIVA VOCE Q&A

Key Learnings: Implementing event-driven programming on CampusVibe reinforced the asynchronous nature of the browser event loop. Separating visual presentation from dynamic behaviors via `addEventListener` guarantees clean, maintainable architecture. Using `e.preventDefault()` on form submissions and right-clicks illustrated how client-side applications can intercept default browser behaviors to deliver bespoke UX.

Q: Bubbling vs Capturing?

Bubbling propagates upwards from target to root; Capturing trickles down from root to target. Defaults to bubbling.

Q: Why preventDefault()?

Cancels native browser actions like form HTTP reloads or context menu openings, allowing custom JS execution.